



ClimatePros

Stream 1 Unified Architecture

A Auto Dispatch · B Job Summarisation Copilot · C Parts Finder

Version 1.3 — Draft for client review · Confidential — ClimatePros / techjays

Companion to the Stream 1 Scope Document — the scope defines *what* is built; this document defines *how*.

Document control

Item	Detail
Document	Stream 1 — Unified Architecture
Version	1.3 — Draft for client review
Prepared by	techjays
Classification	Confidential — ClimatePros / techjays
Supersedes	The two per-author drafts of v1.0 — merged, corrected, and condensed into this single document. v1.2 addressed the round-1 specification review (2026-07-31). v1.3 closes the round-2 tech-lead specification review — the FND-050 blocker and 25 major findings — per the Changelog at the end of this document. The companion Scope Document is unchanged; where architecture and scope disagreed, the architecture was changed to match the scope.

How to read this document

- Sections are numbered 1–12; cross-references appear as §N — for example §6 Core Logic.
- **Cloud placement rule:** anything AI-specific runs on Azure; everything else runs on AWS with the existing platform; models stay on-prem on ClimatePros hardware. The AI request itself makes a single AWS → Azure hop; every B/C model call then travels a second, Azure → on-prem path from the FastAPI service to the LiteLLM gateway on ClimatePros hardware (§4).
- **SQL Server is reached only through the .NET APIs** — neither the dispatch workers nor the AI server holds a SQL connection: one write path, one auth boundary, one place to audit.
- **Use Case A involves no LLM** — a deterministic, explainable ranking. B and C are AI-assisted; the technician can always override. Open items needing ClimatePros confirmation are collected in the Appendix.

Approval

Approved for ClimatePros: _____Date: _____

Approved for techjays: _____Date: _____

Contents

01 Stream 1 Overview	5
A — Auto Dispatch · finding the right technician	5
B — Job Summarisation Copilot · closing out the job	5
C — Parts Finder · finding the part that isn't on the truck	5
02 High-Level Architecture	6
2a · Unified system diagram	6
2b · Detailed architecture per use case	8
03 Technology Stack	10
04 Infrastructure	11
Existing — reused as-is	11
New — introduced by this project	12
Environments	12
Compute sizing	13
Network topology	13
05 Operational Flows	14
A — Auto Dispatch · assignment loop	14
B — Job Summarisation · summarise, review, learn	15
C — Parts Finder · find & allocate	16
06 Core Logic (per use case)	17
A — Auto Dispatch · ranking algorithm	17
B — Job Summarisation · summarise pipeline & retrieval	18
C — Parts Finder · matching & sourcing logic	18
07 Data & Schema	20
A — ElastiCache for Redis structures	20
B — the AI MongoDB collections	20
C — data model	21
08 API Reference	21
A — Auto Dispatch · endpoints	22
B — Job Summarisation · endpoints	22
C — Parts Finder · endpoints	23
09 Observability & Evals	23
Unified observability	24
Evals — the feedback loop (B)	24

10 Cost Projection	25
Cost formulas	25
Unified monthly cost by scenario	25
Database & cross-cloud egress	26
11 Onboarding & Fault Tolerance	26
How every rollout runs	26
A — Auto Dispatch	27
B — Job Summarisation	27
C — Parts Finder	28
12 Non-goals & PII	29
Non-goals	29
PII inventory & data handling	29
Appendix — Open Items	30

01 Stream 1 Overview

ClimatePros runs commercial refrigeration and HVAC field service: ~900–1,000 technicians, 46 branches, 10,000+ sites. Work orders arrive automatically; everything after that still runs on a phone call and a technician's memory. Three use cases, three bottlenecks, one job.

Where AI sits: Use Case A is a deterministic algorithm — no LLM, so every assignment is explainable and repeatable. Use Cases B and C use AI, and the technician can always override it.

A — Auto Dispatch · finding the right technician

Web portal · Dispatcher · deterministic — no LLM

A dispatcher has a job ready and ~40 candidate technicians. The four deciding factors — how far, how familiar, how soon free, how urgent — sit behind separate clicks, and confirming one means dialing candidates one at a time, ~2–3 minutes each. Auto Dispatch ranks on those factors and cascades an accept/decline offer down the shortlist.

B — Job Summarisation Copilot · closing out the job

Mobile · Technician at end of job · AI — technician overrides

Every job ends with a mandatory status change: two free-text notes (customer and office) plus compliance forms — done properly, 10–15 minutes in the field. Thin notes make billing and routing harder and leave the next visit without history.

The Copilot drafts both notes from the technician's rough input, suggests likely form values from similar past jobs, and flags what is missing; the technician reviews, fixes, and submits. Unknown or missing sections degrade to an explicit MISSING placeholder rather than invented text — enforced by the §9 groundedness eval (FND-024) — and it never blocks Save. Target: cut closeout time by up to roughly half — from 10–15 minutes toward the 5–8 minute range, matching the scope document's Success Measure for B.

C — Parts Finder · finding the part that isn't on the truck

Mobile · Technician on-site · AI — technician overrides

On site, the technician needs a part. Three sources the app already answers — truck, nearby technicians, warehouse; the fourth, a third-party supplier, is a phone call, and the same part carries a different number at every vendor. Miss it and it's a return dispatch. Parts Finder suggests the likely part with AI and shows, in one search, where it is across all four sources. No price ranking, no ordering. One working day, three assists: (A) the right technician → (C) the missing part → (B) the closeout note (§5).

02 High-Level Architecture

2a · Unified system diagram

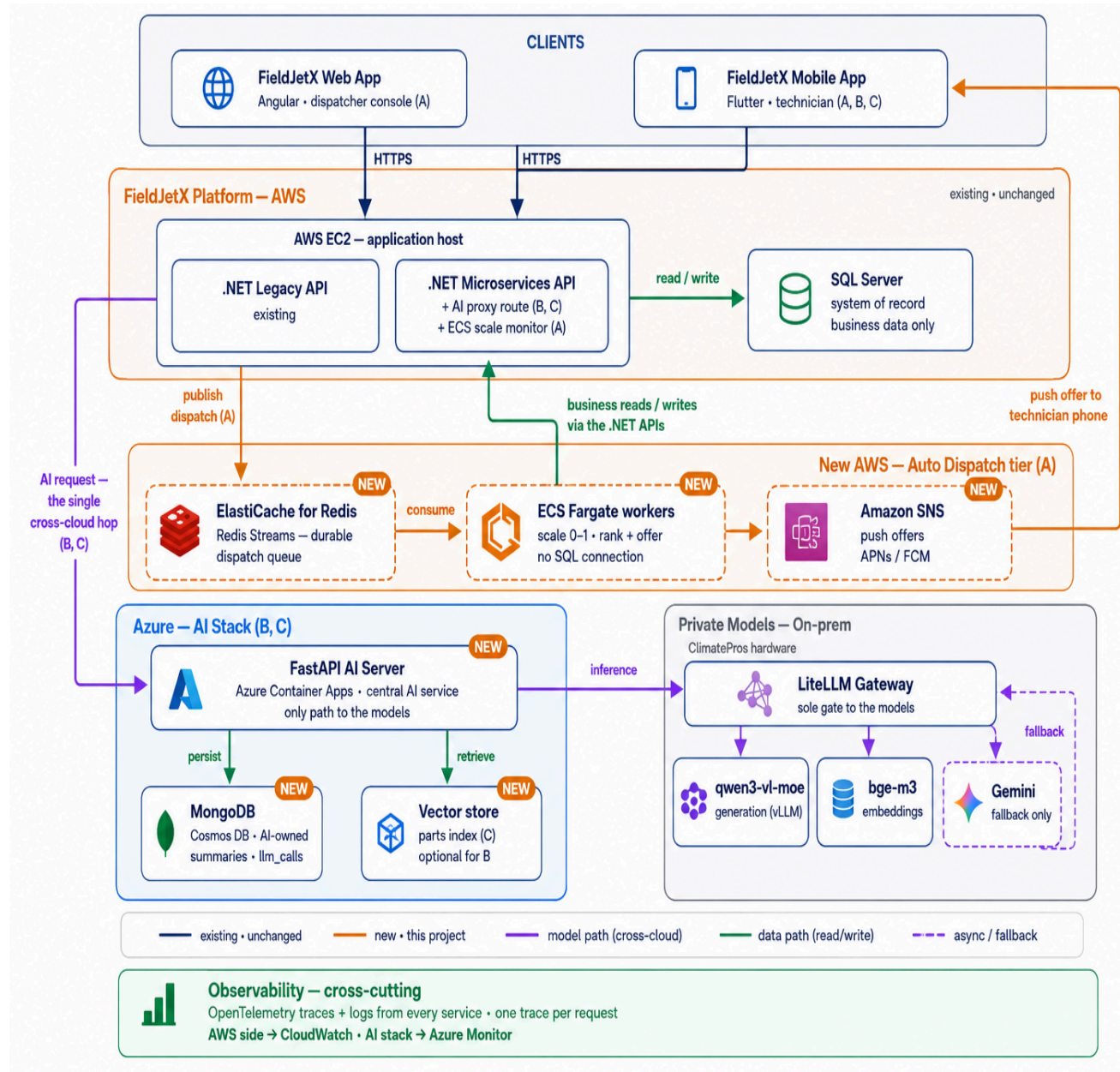


Figure 1 — Unified system diagram · six zones. Workers and the AI server hold no SQL connection; all business data flows through the .NET APIs.

Everything runs on AWS alongside the existing FieldJetX platform — except the AI stack (FastAPI AI server, its own MongoDB, vector store), which runs on Azure, and the models, which stay on-prem. Five relationships carry the whole system:

- **Entry — single front door.** Web and mobile apps speak only to the existing .NET APIs on EC2 over HTTPS. The platform gains exactly two additions: an AI proxy route (B, C) and an ECS scale-monitor route (A).
- **Data — one gate to SQL Server.** SQL Server stays the system of record, read and written only by the .NET APIs on EC2. The ECS dispatch workers exchange all business data — candidates, ranking inputs, assignments — through those APIs, and the AI server owns a separate MongoDB, receiving business context in the request payload. Neither ever connects to SQL Server.
- **Dispatch (A) — entirely AWS, no model.** The .NET API publishes the job to ElastiCache for Redis; the scale monitor spins ECS Fargate 0→1; workers rank technicians and push accept/decline offers through Amazon SNS. The reply returns via the .NET API.
- **AI (B & C) — one AWS-to-Azure hop.** The .NET AI proxy forwards the request, job context in the payload, to the FastAPI AI server on Azure — the single central AI service and the only path to the models.
- **Models — private, on-prem.** All inference goes through the LiteLLM gateway to the self-hosted qwen3-vl-moe (vLLM) and bge-m3 embedding, with Google Gemini available only as an optional external fallback that is off by default in the privacy-first configuration (§12). Every service emits OpenTelemetry traces and logs — CloudWatch on the AWS side, Azure Monitor for the AI stack.

2b · Detailed architecture per use case

A adds: durable dispatch queue + ECS Fargate worker pool + push-offer cascade. The workers consume the Redis stream, coordinate through per-technician locks, and perform every business read and write — including the safe assignment step — through the .NET APIs, which remain the only holder of the SQL connection.

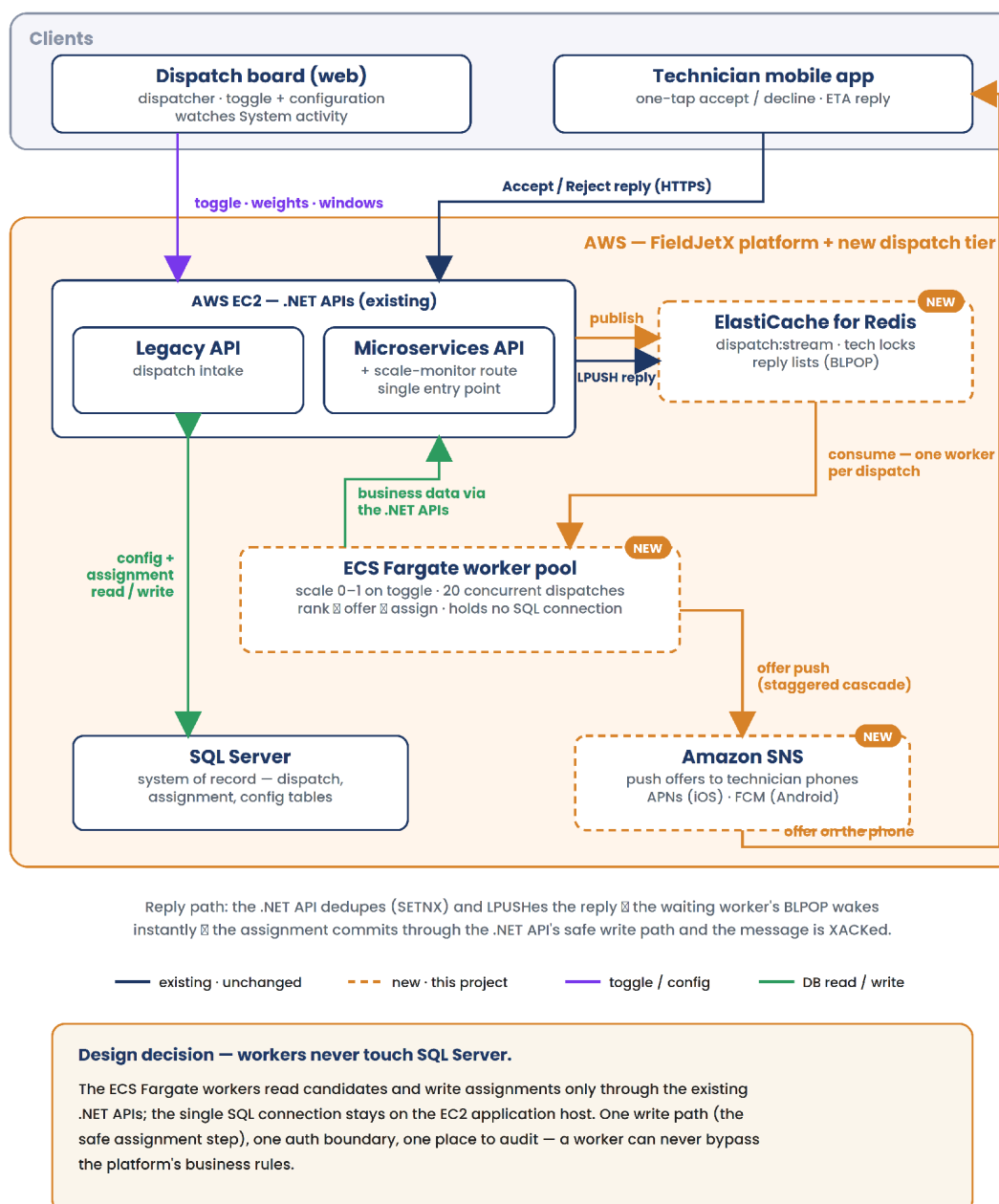


Figure 2 — Use Case A detailed architecture. Entirely within AWS; the worker pool holds no SQL connection.

B & C add: the central FastAPI AI server plus a vector index. B runs the summarise pipeline; C the four-source part search with fuzzy matching — same wiring, different pipeline (§6). The AI datastore is MongoDB on Azure; the vector history index for B is optional, added only if evals demand it.

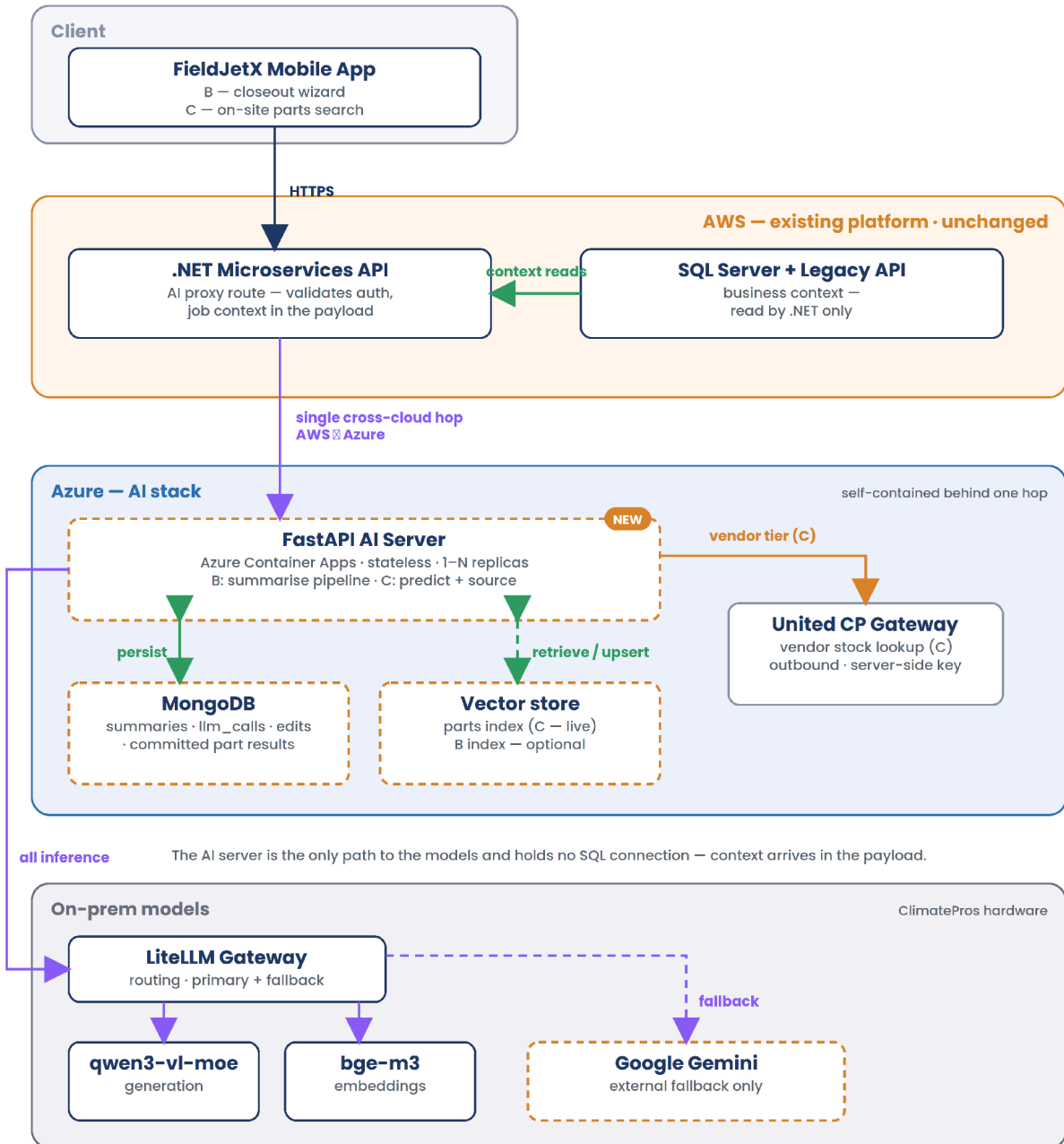


Figure 3 — Use Cases B & C share one AI service; only the pipeline inside it differs.

03 Technology Stack

The existing platform pieces — client apps, the .NET API host, SQL Server, observability — are shared and reused as-is. A's new tier stays on AWS beside the platform; B and C's new tier is the one part of Stream 1 on Azure, reached by a single AWS-to-Azure hop; from Azure, every model call travels the Azure-to-on-prem model path to the LiteLLM gateway (§4).

Layer	A • Auto Dispatch	B • Job Summarisation	C • Parts Finder
Client	Web dispatch console + technician offers (mobile)	Mobile close-out wizard (Flutter)	Mobile on-site search (Flutter)
App / API host	.NET Legacy + Microservices APIs on AWS EC2 (existing) — one shared host; gains the AI-proxy route (B, C) and the ECS scale-monitor route (A).	← shared	← shared
Central AI service	N/A — no LLM	FastAPI on Azure Container Apps (new) — single entry point for B and C; stateless; only path to the models.	← shared
Workers / broker	ECS Fargate pool (0–1 task per enabled board) + ElastiCache for Redis Streams (new)	N/A	N/A
Vector store	N/A	Optional similar-incident index — only if evals demand it	Dispatch-parts history index (live)
Push	Amazon SNS — APNs / FCM (new)	N/A	N/A
Database	SQL Server on AWS — system of record,	← shared	← shared

Layer	A · Auto Dispatch	B · Job Summarisation	C · Parts Finder
	read/written only by the .NET APIs on EC2; workers and the AI server hold no SQL connection. B and C persist AI output to the AI server's own MongoDB on Azure.		
Models	N/A	LiteLLM gateway · self-hosted qwen3-vl-moe (vLLM) · bge-m3 embeddings · Gemini fallback — on-prem, client hardware; any model change must pass the eval set first.	← shared
Observability	OpenTelemetry + CloudWatch (AWS) + Azure Monitor (AI stack) — every service emits traces and logs (§9).	← shared	← shared

Travel time & geocoding (FND-050/051) — the travel-time provider for A is Azuga fleet telematics: traffic-aware driving time/distance taken from the existing, already-contracted nearestvehicles integration that manual dispatch uses today (§4, §6, §10) — per-lookup marginal cost ≈ \$0. Google is used for geocoding only, never travel time. [Azuga contract coverage of the auto-dispatch call volume — to confirm, Open Item #11.]

Model identity (FND-005) — the generation model is pinned by exact tag/revision at the LiteLLM gateway: <QWEN3-VL-MOE EXACT TAG / REVISION — to confirm>; the full serving facts and measured capacity are specified in §4 Capacity (Open Item #1).

Dispatch worker stack — A

Python 3.11+ on an asyncio event loop — fully async, no blocking calls. **Push:** aioboto3 against the SNS Publish API, IAM-role credentials. **Messaging:** redis.asyncio — tasks in via

Redis Streams (XREADGROUP), replies in via per-technician BLPOP. **External calls:** httpx with pooled async connections — all business data via the .NET APIs.

Central AI service stack — B · C

FastAPI + Uvicorn with pydantic schemas. **Model access:** httpx to the LiteLLM gateway — generation on qwen3-vl-moe, embeddings on bge-m3, Gemini fallback handled by the gateway. **Retrieval:** async client to the Azure vector store; cosine similarity over bge-m3 (1024-dim) embeddings — bge-m3 is the pinned embedding model (Appendix). Runtime libraries are version-pinned in a lockfile, and the self-hosted model/tooling licences (qwen3-vl-moe, vLLM, bge-m3) are confirmed for commercial deployment.

04 Infrastructure

Existing — reused as-is

- FieldJetX Mobile App (Flutter, B/C) and Web App (dispatcher console, A).
- AWS EC2 host running the .NET Legacy and Microservices APIs — gains the new routes for A/B/C.
- SQL Server on AWS — system of record, business data only, reached only from the EC2 APIs.
- **Azuga fleet telematics (existing)** — GPS and traffic-aware driving time/distance for the fleet. Auto Dispatch (A) reuses the driving time/distance already returned by the nearestvehicles response the manual dispatch flow uses today, so no new metered travel-time API is introduced and the per-lookup marginal cost is $\approx \$0$ (§6, §10). Google is used for geocoding only. [Contract coverage of auto-dispatch call volume — to confirm, Open Item #11.]

New — introduced by this project

- **ElastiCache for Redis** (AWS) — Redis Streams as the durable dispatch queue (A).
- **ECS Fargate workers** (AWS) — the scaling unit is one task per enabled division/board: each board toggled ON scales its own task 0→1, so total running tasks = boards ON [scaling-unit decision to confirm, Open Item #12]. Workers consume the queue; business data via the .NET APIs.

- **Amazon SNS** (AWS) — outbound push to technician phones (APNs / FCM).
- **Azure Container Apps — FastAPI AI server** — central AI service for B & C; the only component that reaches the models; also serves the observability/evals dashboard via the .NET proxy.
- **MongoDB (Azure Cosmos DB, vCore)** — AI-owned datastore: summaries · summary_edits · llm_calls · part_predictions — §7 is the authoritative collection list (FND-022). No SQL Server coupling.
- **Vector store (Azure)** — dispatch-parts index (C); optional similar-incident index (B), only if evals demand it.
- **LiteLLM gateway + self-hosted models** (on-prem) — qwen3-vl-moe (vLLM) + bge-m3. Google Gemini is wired as an optional managed fallback but is off by default — enabling it requires a documented ClimatePros data-use decision (Open Item #9); when disabled, no data can reach it.

Environments

Environment	Purpose	What differs from Prod
Dev	Integration and feature work	Lowest replica caps; Gemini-only models (no on-prem link) — permitted only because Dev data is de-identified; points at a de-identified / synthetic dataset derived from SQL, never raw production data (FND-041) [Dev dataset build + Dev model decision — to confirm, Open Item #13]
Staging	Pre-prod validation and evals; mirrors Prod topology	Smaller replica ranges; restricted SQL replica; full model stack for eval fidelity
Prod	Live — 900–1,000 technicians, 46 branches	—

Each environment is fully isolated — separate queues, databases, indexes and keys; lower environments never point at live prod data (§12). The Dev dataset is de-identified or synthesised before load — a production SQL snapshot is production data and never enters Dev as-is — and any Gemini processing of ClimatePros-derived data in non-production requires the Open Item #9 DPA basis (region, retention, legal basis).

Alternatively Dev runs a small quantised local model [decision to confirm, Open Item #13].

Compute sizing

Component	Instance size	Scaling	Notes
Dispatch workers (ECS Fargate) — AWS	0.5 vCPU / 1 GB	0–1 task per enabled board (toggle)	60+ concurrent dispatches per instance
AI server (FastAPI) — Azure	0.5 vCPU / 1 GB	2–N on gateway queue wait (§8 admission control)	Min 2 replicas — always-on, no cold start; tolerates instance loss (§11B); §10 floor re-derived
ElastiCache for Redis — AWS	Managed tier	Fixed; sized to queue depth	Persistence on; no-eviction policy
Vector store — Azure	Managed / index-sized	Grows with parts corpus	Sizing pends the §7C seeding decision (FND-011); the earlier 3–4K figure is superseded [record count — to confirm, Open Item #3]
MongoDB (Cosmos vCore) — Azure	M10 (2 vCore / 8 GB)	Fixed tier; manual step-up	AI stack's own store

Network topology

All existing and new non-AI infrastructure sits in one AWS VPC: EC2 and ECS Fargate in an apps subnet, ElastiCache and SQL Server on private subnets with no public IP. The only public entry is the .NET API behind a load balancer / WAF. A separate Azure VNet holds only the AI stack, with MongoDB and the vector store on private endpoints; technician phones never reach either side directly. The third zone is the on-prem model tier — the LiteLLM gateway and vLLM on ClimatePros hardware — reached only from the Azure VNet (FND-038).

- **Seam 1 — AWS→Azure (the AI request)** — the .NET proxy on AWS calling the FastAPI server on Azure. Target state is a site-to-site VPN or VPC-to-VNet peering; the

interim fallback is a locked-down public HTTPS endpoint (IP allow-list / mTLS). The pilot posture must be decided before build (FND-004, Open Item #6): if VPN/peering is in place for the pilot, the interim option is deleted and §12's never-internet-reachable claim holds absolutely; if the interim endpoint is used, that is the documented §12 posture — with compensating controls (WAF, per-source rate limits, client-certificate rotation, alerting on non-allow-listed source attempts) and a dated migration deadline and owner (<DATE / OWNER — to confirm>) so it does not become permanent. [Pilot posture to confirm.]

- **Seam 2 — Azure→on-prem (the model path, FND-038).** Every B/C model call crosses a second boundary: FastAPI → LiteLLM gateway + vLLM on ClimatePros hardware, carrying the full (scrubbed, §12) prompt on the latency path of every request. The on-prem tier is a named third zone with its own connectivity row — mechanism: <ExpressRoute / site-to-site VPN / authenticated private endpoint — to confirm, Open Item #1>; bandwidth and round-trip latency: <VALUES — to measure>; authentication: gateway credentials from Key Vault, mutual TLS on the link; failure behaviour: B and C degrade to their manual paths (§11), A is unaffected; monitoring: the Azure→on-prem hop is a distinct trace span with its own alert (§9); operational ownership: ClimatePros infrastructure for link and hardware, techjays for gateway configuration <to confirm>. Its round-trip is budgeted inside the B/C P95 targets (§9).
- **If the link goes down,** B and C fall back to their manual paths; Auto Dispatch (A) is unaffected — it never touches Azure.
- **Latency and egress** are paid once per B/C request, not per data row — budgeted into the P95 targets (§9) and the egress line (§10).

Provisioning as code — FND-039

All three environments across both clouds are created by Terraform — one workflow and one remote state (with state locking) spanning the AWS VPC, the Azure VNet, and every managed resource this document names. The same code deploys every environment; only per-environment variable files differ (replica caps, tiers, endpoints). Secrets are never in state or variable files: Terraform references Azure Key Vault entries and AWS IAM roles by name, and values are resolved at deploy time (§11, §12). The §11 Phase-0 gate is restated accordingly: terraform apply succeeds and every health check passes.

Capacity & peak-concurrency validation

The scope document assigns peak-concurrency validation against real volume to this deliverable. Static sizing (above) is the starting point; the numbers below are validated by load test in Staging before pilot widening.

- **Dispatch (A)** — load is bounded — ~700 new dispatches/day across ~900–1,000 technicians and 46 branches. One ECS Fargate worker handles 60+ concurrent dispatches (up to 20 in-flight cascades × short offer windows), and the pool scales 0→1 task per active division/board — one task per enabled board, total tasks = boards ON [scaling-unit decision to confirm, Open Item #12] — so the validation target is the busiest branch's morning arrival burst, not the daily total.
- **AI (B/C)** — each call is a single short request (~6–7 s LLM, <200 ms deterministic). FastAPI scales 2→N on gateway queue wait — not raw request concurrency — so the autoscaler cannot respond to gateway saturation by adding pressure to a fixed-capacity bottleneck (FND-016, §8); the shared ceiling is the self-hosted qwen3-vl-moe / bge-m3 gateway, whose measured max-sustained-requests/minute (below) is the input to this load test. Sizing is validated by a staged load test on the full model stack at projected peak closeout + parts-search concurrency.

Model identity & serving facts (FND-005) — the shared ceiling is fully specified, not implied. Generation model: <QWEN3-VL-MOE EXACT TAG / REVISION — to confirm>, <TOTAL PARAMETER COUNT — to confirm> total / <ACTIVE PARAMETER COUNT — to confirm> active parameters, quantisation <SCHEME — to confirm>. Hardware: <GPU MODEL × COUNT — to confirm>, <VRAM — to confirm>. Serving: vLLM max-model-length <VALUE — to confirm> and max-concurrent-sequences <VALUE — to confirm>. Measured throughput: <TOKENS/SEC at B's prompt sizes — to measure>, giving a max sustained <REQUESTS/MINUTE — to measure>. That max-sustained-requests/minute figure is the single number the §4 load test validates against, the §10 capacity statement cites, and the §8 admission-control semaphore (FND-016) is sized to. Embeddings: bge-m3, 1024-dim (pinned, Open Item #8). All placeholders are Open Item #1 until measured on ClimatePros hardware; the ~6–7 s latency, ~10 s timeout and 1→N scaling assumptions are re-validated against them.

- **Exit gate** — a documented Staging load-test run at ≥ projected peak, with P95 within the §9 provisional targets (FND-023), is the gate before Phase-3 expansion beyond the pilot branch.

Backup & disaster recovery

Recovery targets scale with how rebuildable each store is; only the AI MongoDB holds original records.

- **AI MongoDB (Cosmos vCore)** — the only store with original, non-rebuildable records (summaries, summary_edits, llm_calls). Point-in-time backup enabled, ≥ 30-day retention; target RPO ≤ 1 h, RTO ≤ 4 h (to confirm with ClimatePros); restores rehearsed in Staging.
- **Vector store (C)** — derived and rebuildable from SQL Server — no backup; recovery is a re-embed / re-index run (RPO n/a, RTO = re-index time).
- **ElastiCache for Redis (A)** — ephemeral cascade state only; loss is non-critical — in-flight offers surface to the dispatcher (§11), and the durable assignment-decision record (§7A, FND-035) is written through the .NET API, not held in Redis. No backup.
- **SQL Server** — unchanged — ClimatePros' existing backup/DR policy governs the system of record.

05 Operational Flows

A — Auto Dispatch · assignment loop

A new job enters the queue only for divisions with Auto Dispatch on, then works down a ranked shortlist until a technician accepts or the shortlist runs out. The reply-wait timeout stays below the technician-lock TTL, and the worker renews the lock on a heartbeat while a cascade is in flight (§7A), so a lock is not lost mid-flight in normal operation; if a paused worker does lose it, the fence token (§7A, FND-009) rejects its stale write at the assignment endpoint. On timeout the offer simply advances to the next ranked technician.

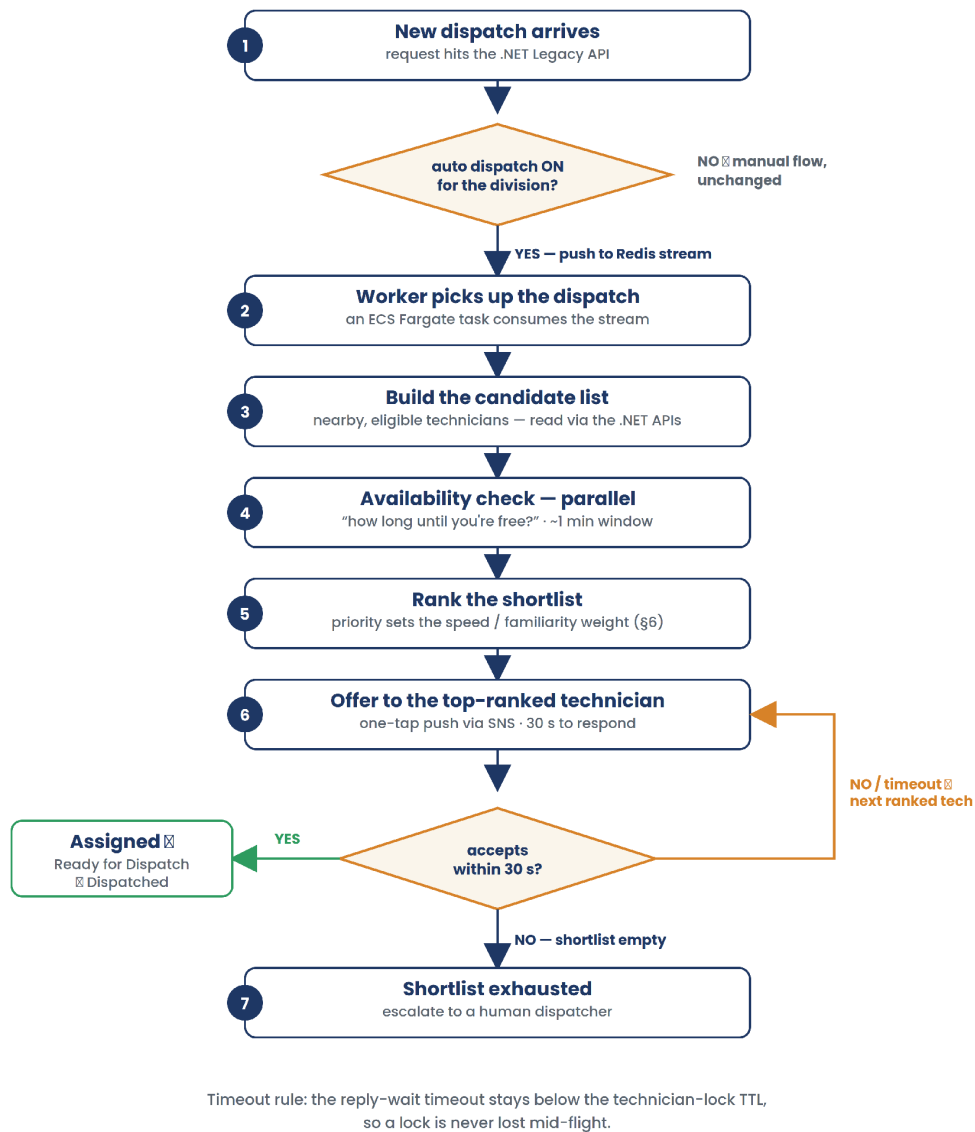
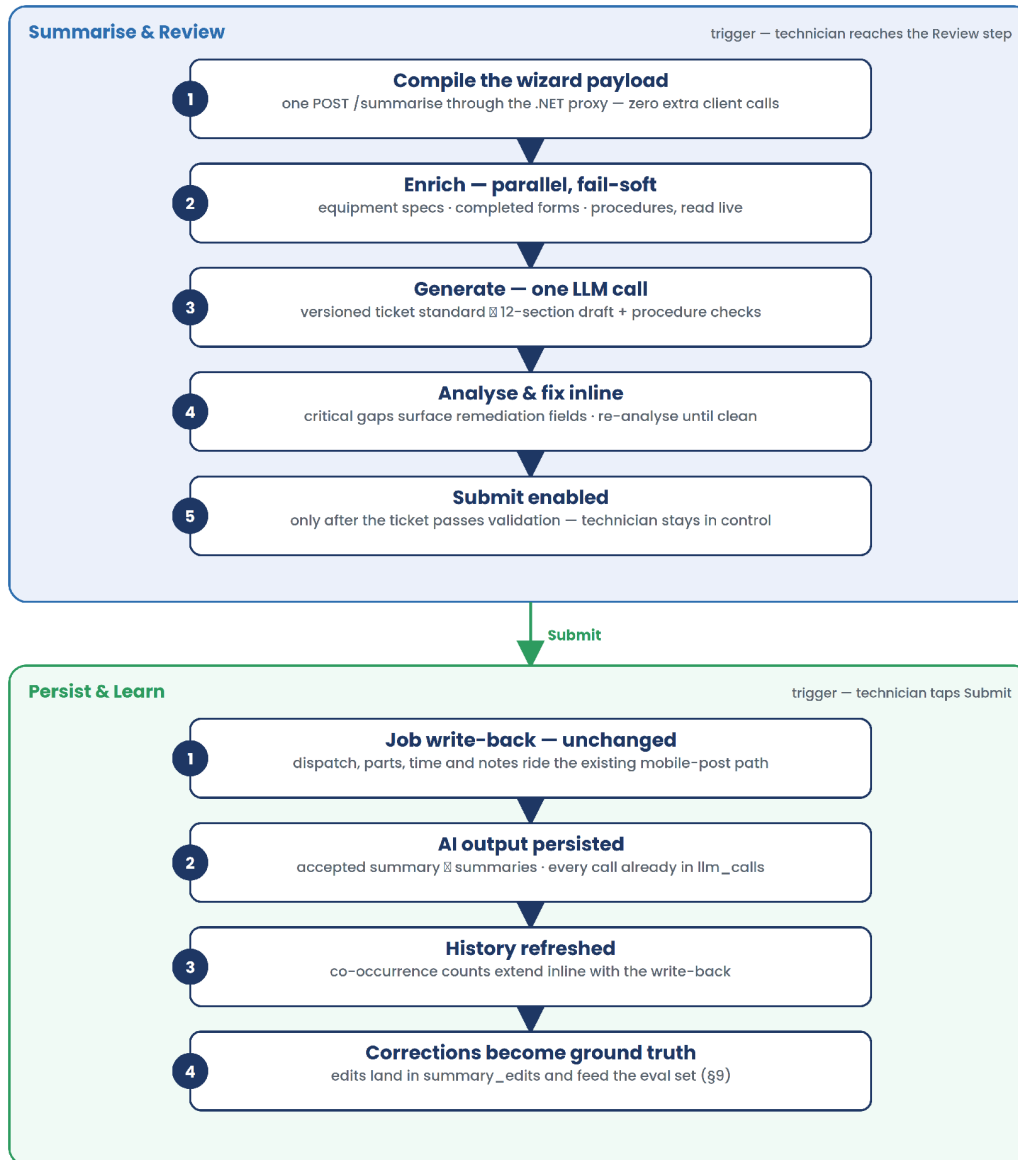


Figure 4 — One dispatch, trigger to outcome: ranked cascade, 30-second offer window, human escalation.

B — Job Summarisation · summarise, review, learn

One POST through the .NET proxy carries the whole wizard payload; a single LLM call drafts the 12-section ticket plus procedure checks; gaps re-analyse until clean. On Submit, the write-back rides the existing path unchanged, AI output persists to the AI MongoDB, and edits become eval ground truth (§9). A site briefing at assignment time is out of scope now.



The flywheel: every clean ticket sharpens the history the next technician's ticket draws from.
The AI drafts; the technician reviews, edits and submits — nothing is invented, and Save is never blocked.

Figure 5 — The two loops the copilot runs: Summarise & Review at the Review step, Persist & Learn on Submit.

C — Parts Finder · find & allocate

For every requested part, the Source API returns every place it can be collected, free own stock first; the needed quantity splits greedily down the sorted list until met. Prediction (which parts the job likely needs) runs before sourcing — the pipeline is §6.

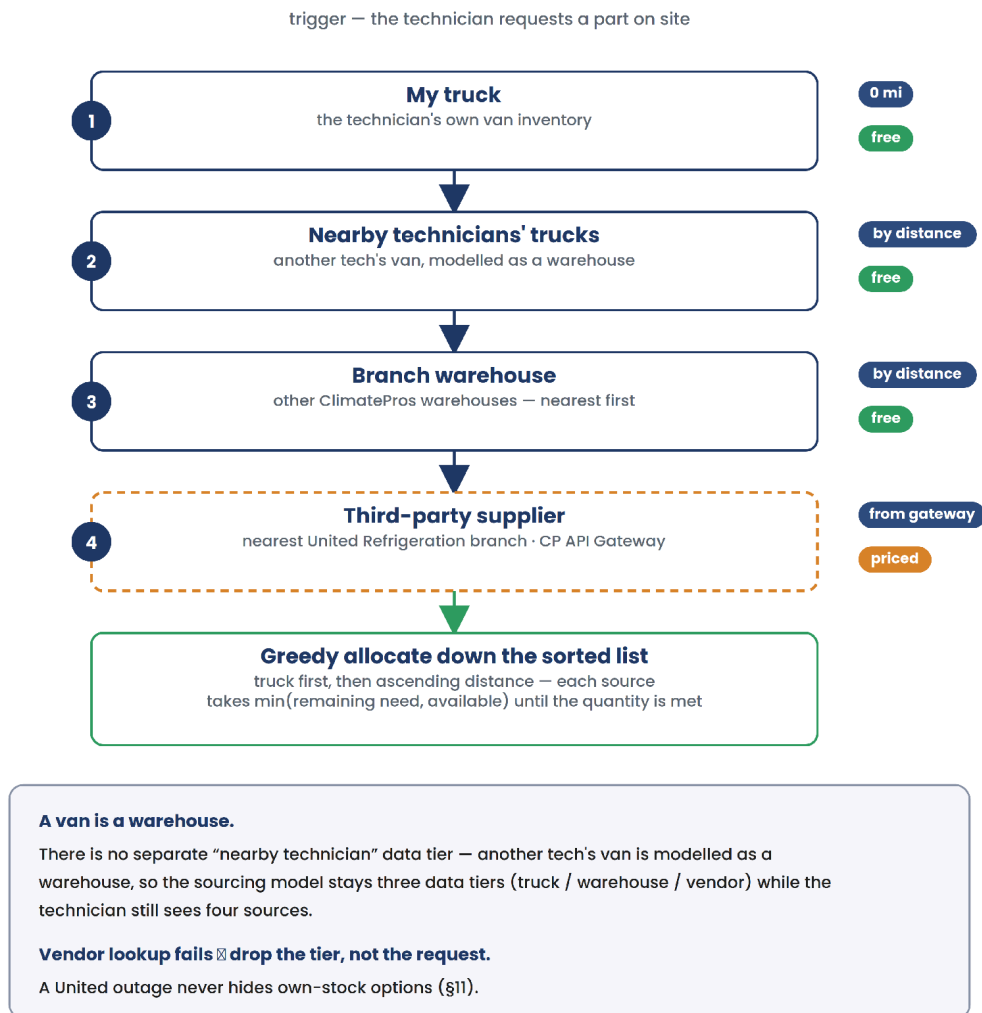


Figure 6 — Four sources, ordered so free stock comes first; the vendor tier is paid and dropped independently on failure.

06 Core Logic (per use case)

A — Auto Dispatch · ranking algorithm

The detail behind “rank the shortlist” in §5 — availability, distance, and visit history collapse into one ordered list. One formula covers every case; a single weight w , chosen from the dispatch type, tunes its behaviour.

Inputs: $free_i$ — minutes until technician i is free; $dist_i$ — miles to the jobsite; $visits_i$ — prior visits to this customer; $dispatchType$ — Emergency · High · Medium.

Step A — unify availability + distance into one ETA

```
travel_i = traffic-aware driving time from Azuga (nearestvehicles response)
# fallback: plain straight-line distance, flagged on the ranking output (FND-051)
eta_i    = free_i + travel_i          # minutes until on site — lower is better
```

Travel-time provider & degraded mode (FND-050/051) — $travel_i$ comes from Azuga fleet telematics: the ranking reuses the driving time/distance already present in the existing Azuga nearestvehicles response — the same integration current manual dispatch uses — so no new metered travel-time API exists on this path and the per-lookup marginal cost is $\approx \$0$ (§10). Google is used for geocoding only, never travel time. Degraded mode mirrors the pattern C already uses in sourcing: traffic-aware travel time when available, plain straight-line distance as the accepted fallback, with the ranking output flagged whenever the fallback was used — the flag is persisted in the assignment-decision record (§7A, FND-035). Controls: a per-lookup timeout $\langle VALUE \rangle$ — to confirm, sub-second class>; a cache of technician-to-site travel times bucketed by time-of-day for repeated pairs (hit-rate $\langle RATE \rangle$ — to measure>); a circuit breaker that trips to the distance fallback after consecutive failures; and a §10 cost-control ceiling that alerts on any metered geo spend (§9). Decision, with rationale: Azuga over a metered maps API because it is already contracted, already on the dispatch path, and supplies vehicle-derived ETAs — a metered API at the same volume would cost $\sim \$4,000/\text{month}$ (§10 derivation). [Azuga contract coverage of the auto-dispatch call volume — to confirm, Open Item #11.]

Step B — weighted score

```
speed_score_i = (eta_max - eta_i) / (eta_max - eta_min)      # lowest eta -> 1.0
fam_score_i   = (visits_i - visits_min) / (visits_max - visits_min)
score_i       = w * speed_score_i + (1 - w) * fam_score_i    # w = speed weight
```

```
# sort score_i descending -> rank 1 = offer target
```

Edge cases: a single candidate is ranked 1 by default. Exact ties resolve by a fixed, deterministic tie-break — data-only, no AI, identical for every dispatch type — so identical inputs always produce an identical order: lowest ETA first; then fewest scheduled upcoming jobs; then the lower technician id. Familiarity already lives in the score, so it is not re-used as a tie-break.

Step C — pick w from the dispatch type

Dispatch type	Strategy	w (speed)	Result
Emergency	Speed-based (locked)	1.0	Pure ETA — familiarity ignored
High Priority	Combined	0.2 (Speed 20 / Fam 80)	Weighted blend; configurable per strategy
Medium Priority	Familiarity-based	0.0	Pure visit count — most familiar wins

One formula, three behaviours — no separate ranking code per type. **Future skill gate:** when technician skill data exists, a job's minimum skill enters as a hard filter on the candidate list, not a score term; the scoring is untouched.

B — Job Summarisation • summarise pipeline & retrieval

One LLM call bracketed by deterministic reads before it and deterministic rules after it — one path, every dispatch. **Inputs:** the wizard answers (reason, parts, time, notes, forms); live context (equipment specs, completed forms, procedures); the versioned ticket_standard.md prompt template; and the SQL co-occurrence history.

- **Compile & forward** — mobile app → .NET proxy (validates auth) → FastAPI, one POST — no extra client round-trips. A deterministic, code-level scrub runs on the FastAPI service before any model call (primary or fallback, on any runtime): customer PII and all pricing — labour/overhead rates, part prices, internal part numbers — are placeholder-substituted out of the prompt and re-inserted into the office-facing output afterward, so the model never sees customer data. The same scrub covers Parts Finder's (C) dispatch-reason text.

- **Enrich, prompt, generate** — parallel fail-soft context reads → one prompt from ticket_standard.md → one LLM call (~6–7 s) → draft 12-section ticket + procedure checks as structured data.
- **Normalise & submit** — draft mapped onto the fixed 12 sections (a missing section becomes an explicit placeholder, never dropped); deterministic rules override sections already known from data; the technician fixes remaining gaps inline; submit writes through the existing path.

Model-call contract — each B/C model call has a hard request timeout (LLM ~10 s ceiling over the ~6–7 s typical; embeddings ~2 s) and a max-token ceiling; structured output is enforced primarily by a JSON schema / constrained decode at the gateway, with the regex-extraction + MISSING-placeholder path (§11) as the fallback, so the app always receives exactly 12 sections.

Voice capture (declared beta) — dictation into the notes field is out of the Phase-1 default path — it ships only behind a flag and only after the written cost-and-privacy assessment is reviewed with ClimatePros (scope B1). Typing stays primary; a transcription failure never blocks submit.

Need	Mechanism
Part / form-value suggestions	SQL co-occurrence: (equipment type × problem code) → most-used parts, falling back to equipment type alone, then global top parts — one aggregation, fully explainable
Compliance procedures	Always fetched live from the source system, never cached or embedded — compliance-critical and subject to change

C — Parts Finder · matching & sourcing logic

Prediction is retrieval-augmented: find the most similar past dispatches, then rank the parts those dispatches used, weighting closeness over raw frequency so one near-identical job outranks several loosely-similar ones.

- **Asymmetric embedding** — historical reasons embed at index time (document task type); the live dispatch reason embeds per request (query task type); both on the self-hosted embedding model — dispatch text never leaves client infrastructure.
- **Retrieve** — top K = 10 nearest neighbours from the vector store, with cosine distances and each neighbour's partsUsed.

- **Rank, aggregate, select** — convert distance to similarity, gate on a threshold, accumulate a cubed-similarity score per part, normalise to a confidence band, return the top parts and pre-select the confident ones.

```
sim = 1 - cosine_distance
if sim > 0.55:                                # SIM_THRESHOLD - neighbour contributes
    for part in neighbor.partsUsed:
        score[part] += sim ** 3 # closeness > frequency
        timesUsed[part] += 1
confidence(part) = clamp(round(score[part] / max_score * 95), 20, 95)
return top 6 parts by score                    # pre-selected if confidence >= 60
```

Constant	Value	Effect
TOP_K	10	Neighbours retrieved per query
SIM_THRESHOLD	0.55	Weaker matches are ignored
Weight	sim^3	Sharply favours very close matches
Confidence band	[20, 95]	Normalised to the top score $\times 95$, then clamped
Output / pre-select	Top 6 $\cdot \geq 60$	Returned to the technician; confident parts pre-selected

Indexing & the self-learning loop: idempotent upsert keyed by dispatchNo; rate-limit-aware batching with retry/backoff; when a dispatch completes, its reason and used parts are embedded and upserted — the very next similar job benefits. The response also reports basedOn — how many neighbours cleared the threshold.

Index admission criteria (FND-029) — completions are not indexed unconditionally. A completed dispatch is admitted to the index only if: (1) its reason text exceeds a minimum length and information threshold $\langle \text{LENGTH} / \text{THRESHOLD} - \text{to confirm} \rangle$ — the filter that excludes the ~40% of notes ≤ 40 characters noted in §12; (2) its parts were not subsequently returned, credited, or corrected; and (3) it is not in the C backtest hold-out set (§9). Where the prediction was accepted with no technician edit, the record is excluded or down-weighted so accepted predictions do not carry the same evidentiary weight as independently chosen parts — breaking the self-reinforcing loop of C learning from its own unverified output. The negative signal (a technician deselecting a pre-selected part) is brought forward from Phase 2 where feasible and captured in part_predictions (§7C, FND-022).

Tier	What it is	Distance	Price
truck	The technician's own van inventory	0 mi	free
warehouse	Other ClimatePros warehouses — including other technicians' vans — plus the branch warehouse	nearest first	free
vendor	Nearest United Refrigeration branch (CP API Gateway)	from gateway	priced

A van is a warehouse — three data tiers behind the four sources the technician sees.

Ordering & allocation: sources rank owned-first, then by travel time within each tier — traffic-aware travel time when available, plain distance as the accepted fallback, flagged in the ordering when used. Tier 0 (own truck, 0 mi) is exhausted first, then Tier 1 (other owned stock), then Tier 2 (vendor). A single configurable cross-tier buffer is the one knob that lets the strict order bend: carried as a per-request config field (`sourcing.crossTierBufferMinutes`), its Phase-1 default is “never” (pure owned-first); set to a margin it lets a dramatically closer vendor outrank a much-farther owned source. Own stock carries no price, only the vendor tier is priced (`InventorySupplierXREF.currentcost`); a large need is split greedily — each source takes $\min(\text{remaining need}, \text{available})$ until the quantity is met. A failed vendor lookup drops that tier, never the request (§11).

07 Data & Schema

A coordinates in-flight work through ephemeral Redis structures; its one durable artefact is the assignment-decision record written into the dispatch status fields at cascade end (FND-035, below). B and C persist AI output into the AI server's own MongoDB on Azure, with zero coupling to the business tables; C additionally maintains a derived vector index.

A — ElastiCache for Redis structures

Structure	Key	Purpose
STREAM	dispatch:stream	Durable buffer between the .NET API and the workers — consumer group (XREADGROUP); each message goes to exactly one worker

Structure	Key	Purpose
MUTEX	lock:tech:{techId}	Per-technician lock, acquired in fixed order to avoid deadlock — a technician is never double-offered; TTL self-releases on a crashed worker, is renewed on a heartbeat while a cascade is in flight, and carries a fence token guarding stale holders (FND-009, below)
LIST	resp:{dispatchId}:{techId}	Per-technician reply channel — the API LPUSHes, the worker BLPOPs (instant wake, no polling); EXPIRE 60 s self-cleans

A worker processes up to 20 dispatches concurrently (redis.asyncio) — a blocked BLPOP on one dispatch never stalls the others.

Lock renewal & fencing (FND-009) — while a cascade is in flight the worker renews the per-technician lock TTL on a heartbeat (<INTERVAL — to confirm>, comfortably under the TTL), so a healthy-but-slow worker does not silently lose its claim. Each lock acquisition also issues a monotonically increasing fence token (Redis INCR per technician lock); the worker passes it on every call to the .NET assignment endpoint, which records the highest token seen in the dispatch status fields and rejects any assignment carrying a lower token with 409 stale_fence_token. A worker paused past the TTL by GC, a network partition, or a stalled .NET call therefore cannot complete an assignment a successor already made — the durable guard sits at the one write path, exactly where §7A previously had no state. Exactly-once (§11A) is conditional on this guard. The dispatch status fields — assigned techId, assignment fence token, assignment source, timestamps, and the assignmentDecision document below — are the only SQL schema change A makes (§12), sequenced expand→migrate→contract in the pipeline (§11, FND-052).

Assignment-decision record (FND-035) — one durable record per dispatch (~700/day), written through the .NET API into the dispatch status fields as an append-once assignmentDecision document when the cascade concludes: the full candidate list with each candidate's eta, distance, visit count, computed score and rank; the dispatch type and weight w applied; whether the travel-time fallback was used (§6, FND-051); every offer with timestamp and outcome — including late accepts rejected under the §8A contract (FND-054); and the final assignment or escalation. One write per dispatch

delivers the audit trail for a system deciding who works across 900–1,000 people, the debugging path for ranking complaints, and the tuning signal A previously lacked.

B — the AI MongoDB collections

Collection	Purpose
summaries	The accepted 12-section ticket + procedure-check results per dispatch — keyed by the dispatchId carried in the payload, not a database foreign key
llm_calls	One record per model call: provider, model, tokens in/out, latency, outcome — feeds cost tracking and the dashboard (§9)
summary_edits	Every human correction to an AI-drafted section — the eval ground truth (§9)

Indicative collections — final schema and index design is agreed with the client DBA (Appendix).

Indicative field-level shape (final index design with the client DBA, Open Item #5).

Collections carry a schemaVersion for forward-only migration:

```
summaries    { _id, dispatchId, ticket:{12 sections}, procedureChecks[], model,
               standardVersion, schemaVersion, createdAt }  idx: dispatchId (unique)
llm_calls    { _id, dispatchId, useCase, provider, model, promptScrubbed,
               completionRaw, tokensIn, tokensOut, latencyMs, outcome, scrubOutcome, ts }  #
model = served model tag (§11, FND-053); scrubOutcome audits the §12 scrub (FND-033)  idx:
dispatchId, ts
summary_edits { _id, dispatchId, section, before, after, editedBy, schemaVersion, ts }  idx:
dispatchId, section
```

C — data model

Entity (SQL Server — read only)	Purpose in this system
Dispatch	reason is the prediction input; completion feeds the learning loop
DispatchPartsRequest / Inventory	Parts catalogue and the parts a dispatch requests/uses
InventoryLocationXref	Stock by location — drives own-stock availability in sourcing
InventorySupplierXREF	currentcost — the vendor price applied to the United tier

AI writes land only in the AI server's MongoDB (committed parts lists, acceptance signals, indexing bookkeeping). The vector index holds one record per completed dispatch — id = dispatchNo, the embedded reason text, and partsUsed metadata — **derived and rebuildable**; the relational database remains the source of truth. Because the AI server has no SQL connection, dispatch completion is pushed to it by the .NET side through the AI-proxy route — the endpoint is now specified in §8C (POST /parts/completions, FND-028); the .NET-side trigger mechanism and its owner remain Open Item #4.

Index seeding & scale (FND-011) — “one record per completed dispatch” against §12's 200,000+ jobs per six months is two orders of magnitude above the superseded 3–4K sizing that §4 previously carried — and the difference changes index tier, query latency, re-index duration and the §10 line. The seeding decision is open [to confirm, Open Item #3]: either (a) a filtered subset — reason text above the §6C admission threshold (FND-029) and/or the last <N — to confirm> months, with the resulting record count <VALUE — to confirm> and growth rate <VALUE — to confirm> recorded here once decided — or (b) the full corpus, which re-sizes the store, re-derives the §10 vector-store line, and requires measured query latency and re-index duration at scale <VALUES — to measure>. Index type and parameters (HNSW m / ef-construction / ef-search, or the managed service's equivalent) are pinned once the store is selected (Open Item #3).

part_predictions — C's retrieval intermediates (FND-022): one document written on every /parts/predict call, so the §4 list is reconciled and §7 is authoritative — the AI collections are summaries, summary_edits, llm_calls, and part_predictions. Shape: { _id, dispatchId, reasonHash, embeddingModelVersion, neighbours: [{ dispatchNo, cosineDistance, clearedThreshold }], parts: [{ part, accumulatedScore, confidence, preSelected }], usedParts (populated by the §8C completion push, FND-028), ts } idx: dispatchId, ts. This one write serves three needs at once: debugging a bad prediction (the top-K neighbours, cosine distances, SIM_THRESHOLD gate, cubed-similarity accumulation and confidence are all captured, not just basedOn), the C backtest (§9), and the Phase-2 correction loop (accepted vs deselected parts against usedParts).

08 API Reference

The HTTP surface, kept deliberately brief — endpoints, callers, and purpose only. Request/response schemas live in the accompanying API specification: Streaml-AI-API.openapi.yaml, version 1.0.0 — an OpenAPI 3.1 document covering all 14 routes below (including the new internal /parts/completions, §8C) with request/response

schemas, examples, per-endpoint error codes on the standard envelope, and authentication (FND-014). The FastAPI pydantic models and the .NET typed client are generated from that file, so the implemented schema cannot drift from the contract; this chapter is the index — the OpenAPI file is the contract. The §6C field `sourcing.crossTierBufferMinutes` is specified there as: integer, unit = minutes, range <MIN-MAX — to confirm with ClimatePros>, nullable, default null = “never” (pure owned-first). All existing endpoints are unchanged and out of scope here.

A — Auto Dispatch · endpoints

Method	Path	Caller	Purpose
POST	/divisions/{id}/toggle	Dashboard	Auto Dispatch ON/OFF per division — enable pulls pending work into the stream and scales the pool up; disable drains to zero
PUT / GET	/divisions/{id}/config	Dashboard	Read/update per-division ranking weights and windows — applied to the next loop, no restart
POST	/devices/register	Mobile	Register/refresh a push token — upsert on (techId, token); re-posted on OS token rotation
POST	/technicians/{id}/availability	Mobile	“When will you be free” reply — updates general availability read by ranking
POST	/dispatches/{id}/respond	Mobile	Accept/reject one specific offer — deduped, then LPUSHed to the waiting worker while the offer is still current; otherwise 409 with a stable code (below, FND-054)

Client-facing (mobile/dispatcher) endpoints inherit the platform’s existing rate limiting; the AI routes (B/C) are internal-only, called solely by the .NET proxy. Every endpoint sits behind the platform’s existing auth middleware — admin Dashboard auth on the control plane, technician mobile auth on the data plane (identity must match the techId in the path or body). All endpoints return the platform’s standard error envelope (stable

machine code, human-readable message, correlation requestId) with the usual 400/401/403/404/409/500 semantics.

Late accept / already assigned (FND-054) — /dispatches/{id}/respond first checks whether the offer is still current — the resp:{dispatchId}:{techId} key exists and the 30-second window is open. If not, it returns 409 with a stable machine code on the standard envelope: offer_expired when the window has closed, already_assigned when the dispatch has already been assigned — never an LPUSH onto a key nobody reads that silently expires. Client contract: the push notification carries the offer-window deadline, so the offer card expires locally when the window closes without a round trip; on 409 the card shows “this job went to another technician”. Two technicians can therefore never each believe they hold the same job. Every response outcome, including rejected late accepts, is written to the assignment-decision record (§7A, FND-035).

B — Job Summarisation • endpoints

Method	Route	Purpose	Latency
POST	/api/v1/summarise	Wizard payload → 12-section ticket + procedure checks	~6–7 s
POST	/api/v1/analyse	Ticket → gap findings + inline remediation; re-run until clean	~6–7 s
GET	/api/v1/suggestions	Top-k part suggestions from the co-occurrence tables — deterministic, no LLM	< 200 ms
GET	/admin/dashboard	Edit-rate, latency, cost, parse failures — read from the AI MongoDB	< 500 ms
GET	/health	Liveness probe	< 50 ms

No client calls these routes directly — the .NET proxy is the only caller (internal API key + private VNet); the dashboard is a .NET-proxied route with an admin-role check on the existing Entra token. /summarise and /analyse are read-only and safe to retry.

Admission control (FND-016) — the internal path is the one that matters, so the AI service protects the fixed-capacity gateway itself: a semaphore bounds concurrent gateway calls to the measured FND-005 capacity (<MAX CONCURRENT / REQ-MIN — to measure, Open Item #1>); behind it sits a bounded queue with a maximum wait <VALUE

— to confirm>; beyond that the service answers fast with 429/503 and Retry-After — clients treat these exactly as the §11B/§11C manual fallback, so submission is never blocked. A per-technician and per-submission cap prevents any one client from consuming the ceiling. Container Apps scales on queue wait, not raw request concurrency (§4), so the autoscaler relieves the queue rather than adding pressure to the bottleneck.

C — Parts Finder · endpoints

Method	Route	Purpose
POST	/parts/predict	RAG-predicted parts for a dispatch — via the AI proxy route; 503 when prediction is unavailable (client falls back to manual add)
POST	/parts/source	All sources per confirmed part, grouped by tier with quantity and distance — via the AI proxy route
GET	/health	Liveness / deploy probe — not proxied to clients
POST	/parts/completions	Internal (FND-028): the .NET side pushes dispatch completion — { dispatchNo, reason, partsUsed }; idempotent upsert keyed by dispatchNo; drives the §6C learning-loop upsert subject to the FND-029 admission criteria

The vendor tier calls the ClimatePros CP API Gateway for United Refrigeration server-side — the subscription key never leaves the server. Both routes are read-only, side-effect free, and safe to retry; dispatch list/detail and the manual-add inventory tabs stay on the existing .NET APIs, unchanged.

Completion push (FND-028) — POST /parts/completions is internal-only, authenticated by the same internal-API-key + private-VNet path as the other AI routes. Payload: { dispatchNo, reason text, partsUsed }. The write is idempotent — an upsert keyed by dispatchNo — so the .NET side retries safely with backoff on failure. The .NET-side trigger is either a synchronous call on dispatch completion or an outbox row drained by a scheduled job; the outbox is the more robust option under AI-tier outages [mechanism and owner to confirm, Open Item #4]. A monitor on index growth rate (§9 alert table) detects a broken trigger directly, instead of leaving a silently static index to be inferred later.

09 Observability & Evals

One common chapter for all three use cases. Observability is operational and unified — one tracing/logging layer watches everything the same way. Evals apply only where a model is in the loop: A is deterministic with nothing to eval; B has a production-grounded eval loop; C validates retrieval quality on a held-out set, with a correction-based loop as future work.

Unified observability

- **Distributed traces** — one trace per request across every hop (.NET → Redis → worker → .NET, or .NET → Azure → LiteLLM → model); the AWS→Azure hop is a span like any other, so its round-trip is visible.
- **Structured logs** — one schema; AWS services report to CloudWatch, the AI stack to Azure Monitor.
- **Correlation** — the requestId from the error envelope joins traces, logs, and the AI collections (llm_calls, summary_edits), so a slow trace, a log line, and an eval finding resolve to one id.

P95 targets — provisional (FND-023): set now from the figures this document already contains, labelled provisional-pending-baseline: /summarise and /analyse P95 ≤ 10 s (the §6B hard ceiling over the ~6–7 s typical, inclusive of the Azure→on-prem model round-trip, §4); deterministic endpoints (/suggestions, /parts/predict, /parts/source) P95 ≤ 200 ms; dashboard P95 ≤ 500 ms; A's offer round-trip inside the 30 s offer window, with the ~1 min availability window and BLPOP timeout as outer bounds. The §4 Phase-3 exit gate tests against these provisional targets; they are re-baselined once Azure Monitor has real traffic — the gate no longer depends on targets that only exist after production.

Alert table (FND-023) — every §11 failure mode maps to at least one alert below, so nothing degrades silently; the business-level usage alert is how a silent total outage presents.

Signal	Threshold	Window	Severity	Owner
Parse-failure rate (B)	<THRESHOLD — set at pilot baseline>	15 min	P2	to assign — techjays on-call / ClimatePros ops

Signal	Threshold	Window	Severity	Owner
Model error / timeout rate (gateway)	<THRESHOLD — set at pilot baseline>	15 min	P1	to assign — techjays on-call / ClimatePros ops
Gateway queue depth / wait (§8)	wait > §8 max-wait bound	5 min	P1	to assign — techjays on-call / ClimatePros ops
Redis pending-entry age (A)	> XAUTOCLAIM idle threshold (§11A)	5 min	P1	to assign — techjays on-call / ClimatePros ops
Cosmos RU + storage headroom	< 20% headroom (proposed)	daily	P2	to assign — techjays on-call / ClimatePros ops
AI-proxy 5xx rate	<THRESHOLD — set at pilot baseline>	15 min	P1	to assign — techjays on-call / ClimatePros ops
AI-assist usage cliff (business)	B+C call volume < <X% — set at baseline> of trailing norm	1 h	P1	to assign — techjays on-call / ClimatePros ops
Vector-index growth rate (C, FND-028)	no growth over <N — to confirm> days at normal completion volume	daily	P2	to assign — techjays on-call / ClimatePros ops
Metered geo spend (A, FND-050)	any spend > \$0 on travel-time lookups	daily	P2	to assign — techjays on-call / ClimatePros ops
Served model tag divergence (FND-053)	served tag ≠ pinned tag	5 min	P1	to assign — techjays on-call / ClimatePros ops
Edit-rate drift (existing monitor)	section crosses rolling-baseline threshold	daily	P2	to assign — techjays on-call / ClimatePros ops

Evals — the feedback loop (B)

- llm_calls records provider, model, tokens, latency and outcome for every call — cost and reliability queryable at any time.
- summary_edits records every human correction to an AI section — labelled ground truth from normal usage.

- The eval set is rebuilt from those corrections and re-run before any prompt, ticket-standard, or model change ships.
- **Headline metric:** edit-rate per section, tracked over time and per customer.
- **Guardrail cases** — the eval set includes adversarial / prompt-injection free-text inputs (B's closeout text and C's call-reason text both feed a prompt or matching pipeline); the check is that injected instructions never alter structure, invent facts, or leak scrubbed data.
- **Drift monitor** — edit-rate per section is a thresholded monitor, not just a trend line — a section crossing its rolling-baseline threshold (or a customer-level spike) raises an alert for review, distinct from the headline chart.
- **Groundedness eval (FND-024)** — each claim in each generated section must trace to a specific input field or retrieved context item; scored as a fabrication rate per section, gating release alongside edit-rate — it catches the ordinary plausible-but-invented sentence in a populated section that the adversarial guardrail cases and edit-rate do not. An LLM-as-judge is acceptable only after validation against human labels on a seed set, with the judge model stated in the eval config. Until this eval runs, §1/§12 claim only the mechanism actually in place: unknown sections degrade to an explicit MISSING placeholder.
- **Release gate — arithmetic (FND-025)** — the gate is numeric, not a trend. Edit-rate per section carries a target and a minimum acceptable value, set from the pilot baseline (<TARGET / MIN — set at baseline, Open Item #14>); C's top-3 hit-rate likewise (<TARGET / MIN — set at baseline>). Regression tolerance: no section's edit-rate may worsen by more than 2 percentage points versus the previous release, and the aggregate may not worsen (proposed values; ratify at baseline). The gate fails closed when the eval set holds fewer than 200 items (proposed) — an empty or thin set blocks release rather than passing it. Seeding before go-live: a supervisor or small group hand-corrects a stratified sample of historical closeouts across equipment types and problem codes — §12's 12 months of history makes this cheap — so the eval set is never empty at launch.
- **Independent quality signal (FND-026)** — a falling edit-rate is equally consistent with the model improving and with technicians accepting drafts unread — which cutting closeout time incentivises — so at least one metric that does not move with engagement is reported alongside it: periodic supervisor/QA scoring of a random sample of accepted summaries against a rubric, plus the FND-024 groundedness

eval. Time-on-review-screen per section is tracked, and falling edit-rate combined with falling review time is flagged as disengagement, not improvement. Edit-rate is segmented per technician so a careful minority's corrections are not averaged away.

Prompts build from a versioned ticket_standard.md and every call logs its model id, so any production ticket traces back to the exact standard version and model. The dashboard is served by the same FastAPI container from the AI MongoDB — no separate web app, proxied through .NET with an admin-role check. One caveat: the structured outcome codes in the history are weak signal (problem code OTHER at 38%, summary code SERVICE at 88% of filled rows) — summary_edits works around this by labelling the model's actual output, but the eval set is only as good as what technicians correct.

Evals — retrieval quality (C)

Parts Finder is validated on retrieval quality before rollout and re-validated on any change to the embedding model or index.

- **Dataset** — a held-out set of completed historical dispatches with known partsUsed, drawn from the same corpus (§12) but never used to seed the live index.
- **Metric** — top-3 hit-rate — does a dispatch's actually-used part appear in the top-3 predicted parts (the scope document's C success measure). basedOn (neighbours clearing SIM_THRESHOLD) is reported alongside so low-support predictions are visible.
- **Trigger** — the backtest re-runs before any embedding-model swap, SIM_THRESHOLD / weighting change, or index rebuild ships; a regression blocks the change. A correction-based loop (technician confirms / removes predicted parts → labelled signal) is the planned Phase-2 extension, mirroring B's summary_edits.

10 Cost Projection

One shared infrastructure bill across A/B/C — same compute tiers, same fixed floor. A use case appears only as a load driver (dispatch hours-on, B+C call volume), not as a separate section. Figures are for one environment; Dev/Staging each duplicate the bill.

Pending re-pricing

Azure lines (Container Apps, vector store, Monitor) carry AWS rate-card estimates, not real Azure prices yet; Cosmos DB is a real Azure quote, unverified for region/tier. Re-price before using this for a budget; the headline scenario totals below are provisional and are not a budget figure until Azure list pricing is confirmed (Open Item #2).

Cost formulas

```
# ECS Fargate – dispatch workers (0.5 vCPU / 1 GB)
per_task_hour = 0.5 x $0.04048 + 1 x $0.004445 = $0.024685
monthly_cost = N x H x 30 x per_task_hour      # N = tasks (0-1), H = hours ON/day

# fixed / near-fixed lines
ElastiCache Redis = $0.032/hr x 730 = ~$23.36/mo
SNS push (75k pushes/mo) = ~$0.04/mo
Container Apps (AI) = ~$40+/mo (min 2 replicas, $4/$11B;
~$20/replica-mo at the AWS-sourced rate – Open Item #2)
Vector store = ~$5/mo at the superseded 3-4K sizing –
re-derive per the $7C seeding decision (Open Item #3)
Azure Monitor + OTel = $15-$30/mo
Cosmos DB Mongo M10 = $0.129/hr x 730 = ~$94/mo flat
# Travel time (A) – Azuga fleet telematics (existing contract; FND-050)
# ~40 elements/dispatch x ~700 dispatches/day ≈ 28,000 lookups/day (~840k/mo) pre-cache
# cache: technician-to-site pairs bucketed by time-of-day – hit-rate <RATE – to measure>
travel_time_monthly = ~$0 (rides the contracted Azuga
nearestvehicles response)
# metered alternative (not selected): ~$5/1,000 elements x ~840k/mo ≈ ~$4,000-$4,200/mo –
~20x this cloud bill
```

Unified monthly cost by scenario

Component	Light (2 h/day)	Typical (8 h/day)	Heavy (24×7)
Dispatch workers – Fargate	~\$1.48	~\$5.92	~\$17.77
AI server – Container Apps	~\$40	~\$40–\$50	~\$50–\$60
Redis (fixed)	~\$23.36	~\$23.36	~\$23.36
SNS push	~\$0.04	~\$0.04	~\$0.04
Vector store (flat)	~\$5	~\$5	~\$5
Monitor + OTel	~\$15	~\$22	~\$30
MongoDB – Cosmos vCore (flat)	~\$94	~\$94	~\$94

Component	Light (2 h/day)	Typical (8 h/day)	Heavy (24×7)
Variable — per B+C call	< \$1	~\$2–\$5	~\$10–\$25
Travel time (A) — Azuga (FND-050)	~\$0	~\$0	~\$0
Total / mo (est.)	~\$179	~\$192–\$205	~\$230–\$255

Fixed floor (Redis + vector store + Mongo + Monitor + the min-2 AI-server floor of ~\$40, §4/§11B) ≈ \$177–\$192/mo regardless of traffic; only Fargate, AI-server scale-out beyond the min-2 floor, and the per-call variable scale toward zero. Self-hosted models carry no per-token fee — the marginal cost of a B/C call is a fraction of a cent. Travel time (FND-050) is now a first-class line, carried at ~\$0: A's ranking touches ~40 candidates per dispatch × ~700 dispatches/day ≈ 28,000 lookups/day (~840k/month) before the time-of-day-bucketed technician-to-site cache (hit-rate <RATE — to measure>), and every lookup reads driving time from the already-contracted Azuga nearestvehicles response — the same integration manual dispatch uses — so no metered travel-time API exists on this path. The decision and rationale are recorded in §6; a cost-control ceiling alerts on any metered geo spend (§9). For visibility, the metered alternative would have been ~\$5/1,000 elements × ~840k/month ≈ ~\$4,000–\$4,200/month — roughly 20× this cloud bill — which is why the provider is pinned. [Azuga contract coverage of the auto-dispatch volume — to confirm, Open Item #11.] The vector-store line still reflects the superseded 3–4K sizing; the §7C seeding decision (Open Item #3) re-derives it, and full-corpus seeding would materially raise that line and these totals.

Gemini & on-prem model cost — Google Gemini fallback is \$0 in the default configuration (off). If ClimatePros enables it, budget a per-token line = fallback-trigger rate × tokens/call × Gemini list price — to be modelled once a trigger rate is agreed. On-prem GPU/hardware for the self-hosted models (qwen3-vl-moe, bge-m3) is intentionally excluded from these cloud totals: it runs on existing ClimatePros hardware, and the variable model cost of a B/C call is a fraction of a cent.

Database & cross-cloud egress

SQL Server stays on AWS, unchanged — existing and sunk, no incremental cost and no egress (only AWS-side compute connects to it). The one cross-cloud line is the AWS→Azure AI request/response (B/C): one pair per call, not a per-row transfer. Sizing

needs a traffic estimate — get an AWS Data Transfer OUT quote once real volume is known.

11 Onboarding & Fault Tolerance

Two questions per use case: how it reaches live traffic from an empty environment, and how it behaves when a dependency fails. The rollout shape is identical for all three; deployment rides the client's existing pipelines — Stream 1 adds no new deployment tooling.

How every rollout runs

Phase	Purpose	Gate before the next phase
0 · Provision	Stand up what the feature cannot start without — network, hosting, queues, stores, credentials	terraform apply succeeds (§4, FND-039) and every dependency is reachable and health-checked
1 · Seed	Load what the logic reads on its first request — config, schema, indexes, history	Nothing is empty at first call — no cold start
2 · Roll out	Ship behind a flag or app-version gate, pilot cohort only	Pilot cohort live; everyone else unchanged
3 · Go live	Monitoring first, then one division/branch, then expand	Pilot clean on its own metrics before widening

CI/CD & release management

Deployment rides ClimatePros' existing pipelines — Stream 1 adds no new tooling — but every new component ships through an automated build-test-release path with revision-based rollback.

- **Build & test** — every commit runs lint, unit tests, and integration tests against mocked .NET / gateway / vendor stubs (Dev uses Gemini-only mocks and the de-identified / synthetic Dev dataset — §4, FND-041); the container image is vulnerability-scanned before it can be promoted.

- **Gated promotion** — Dev → Staging (full model stack, eval fidelity) → Prod. For B, the eval set (§9) must pass before any prompt / ticket-standard / model change is promoted — a failed eval blocks the release, it does not merely flag it.
- **Rollback** — all three runtimes are covered (FND-053): Azure Container Apps (FastAPI, B/C) and ECS Fargate (workers, A) are revision-based and revert to the previous healthy revision in one step; the on-prem model tier reverts by re-pinning the previous model tag at the LiteLLM gateway (below). A bad deploy is caught at the health-check / pilot-metric gate before the full cohort sees it, and the feature-flag / min-version gate (§11) reverts client exposure instantly without a redeploy.
- **Secrets & config** — resolved from Azure Key Vault / IAM roles at deploy time (§12) — none in source or images.
- **Data & schema migrations (FND-052)** — a migration stage runs in the pipeline before the app deploy and is gated the same way as code. MongoDB: a versioned, idempotent, forward-only migration runner keyed to the schemaVersion the collections already carry (§7B), with the explicit reader-compatibility rule that a version-N reader must read a version-N-1 document — unknown or missing fields default, never fail. SQL: A's dispatch status fields (§7A/§12 — assigned techId, fence token, assignment source, timestamps, assignmentDecision document) are sequenced expand→migrate→contract, so the .NET deploy and the schema change land independently. Vector index: any rebuild is a dual-index build-then-swap cutover behind the same feature flag as the rollout, so C never serves an empty or half-rebuilt index.
- **Model releases — the third runtime (FND-053)** — model tags are pinned at the LiteLLM gateway, so the served version is declarative and diffable. A model change is deployed by <ROLE — to confirm: ClimatePros infrastructure with techjays sign-off> by updating the pinned tag; the one-step revert is re-pinning the previous tag. Every call records the served model tag in llm_calls (the field exists, §7B), and an alert fires if the served tag diverges from the pinned tag (§9 alert table) — so “any model change must pass the eval set first” (§3) now has a deployment mechanism behind the gate. An embedding-model change is a combined procedure, because it invalidates every vector — it is simultaneously a model release, a full re-embed, and an index cutover: eval gate → build a parallel

index under the new model → run the C backtest against it (§9) → swap behind the feature flag → retain the old index for one rollback window.

A — Auto Dispatch

Onboarding: Redis cluster + consumer group, worker image and scale policy, SNS platform apps (APNs/FCM), IAM roles (Phase 0); per-division weights seeded with the toggle defaulting OFF and an escalation target defined (Phase 1); push-enabled app build with a minimum-version gate — tokens upsert on (techId, token) and re-register on rotation (Phase 2); pilot one division, validate exactly-once assignment and escalation, then expand (Phase 3). A technician without a push registration is skipped to manual — never offered a job they cannot receive — and rejoins on their next token refresh.

New personal data

The push token and its SNS endpointArn are the only new PII this project stores — encrypted at rest, limited to the dispatch backend, overwritten on rotation, cleaned up when dead (§12).

Failure	Behaviour — and why it is safe
Worker dies mid-dispatch	The dispatch stays pending in the Redis stream; the replacement ECS task for that board (one task per board, §4) claims it via XAUTOCLAIM once its idle time exceeds <IDLE THRESHOLD — to confirm; set above the 30-second offer window>. Expected recovery = task-replacement time + idle threshold ≈ <VALUE — to confirm> — the dispatch is delayed, never lost. If concurrent redundancy is wanted instead, raise the floor to two tasks per board and re-derive the §10 Fargate line [Open Item #12]
Worker slow, not dead	Its lock-renewal heartbeat (§7A, FND-009) keeps the per-technician claim alive, so a takeover is blocked — the same job never reaches two technicians. Distinguishable from a crash precisely because the heartbeat continues
Worker truly crashed	The heartbeat stops: the technician-lock TTL lapses, and after the XAUTOCLAIM idle threshold the replacement task claims the pending stream entry. If the “crashed” worker was in fact paused and later resumes, its stale fence token is rejected at the .NET assignment endpoint (§7A) — it cannot complete an assignment a successor already made

Failure	Behaviour — and why it is safe
No technician accepts	The 30-second window cascades down the shortlist, then escalates to a human — a job is never silently dropped

Net effect — exactly-once, conditional on the fence-token guard (§7A, FND-009): every request is processed even if a worker dies, and a stale worker's late write is rejected at the assignment endpoint, so none is processed twice even if a worker is slow or paused.

B — Job Summarisation

Onboarding: private subnet + internal load balancer (no public URL), always-on FastAPI at min 2 instances, model-gateway path and secrets in Key Vault (Phase 0); DBA sign-off on the read-only SQL account, MongoDB collections + indexes, ticket_standard.md v1 frozen, co-occurrence tables seeded from 12 months of history, pilot customers' procedures verified (Phase 1); copilot build + min-version gate — older app versions keep the manual review page and never call the AI route (Phase 2); monitor LLM latency/error/parse-failure and the edit-rate dashboard, pilot one branch, capture the baseline, then expand — any later prompt or model change must pass the eval set first (Phase 3).

Failure	Behaviour — and why it is safe
An enrichment read fails	Pipeline continues without that block (fail-soft) — missing context costs quality, not availability
Private gateway outage / saturation	AI features degrade to the manual experience; calls past the timeout fall back to the plain recap — submission is never blocked
Malformed model output	Parser falls back to regex extraction, then MISSING placeholders — the app always receives exactly 12 sections
A Container Apps instance dies	The load balancer routes to the remaining instance and the replica is replaced — min 2 always-on, no cold start

SQL Server stays the single source of truth — the AI tier writes only to its own MongoDB, so an AI failure can never corrupt or block business data.

C — Parts Finder

Onboarding: Container Apps on the private VNet, vector index created, connectivity to the embedding model, the .NET APIs and the United gateway health-checked (Phase 0); batch-embed historical dispatches (idempotent upsert by dispatchNo) and validate retrieval on a held-out set — the index is never empty at go-live (Phase 1); ship “Add Parts with AI” behind a per-branch feature flag and pilot a small cohort watching acceptance rate and source mix (Phase 2); expand division by division and wire the on-completion re-embed pipeline so the index grows as dispatches close (Phase 3).

Failure	Behaviour — and why it is safe
Models / vector store unavailable	/parts/predict returns 503; the client falls back to manual add — prediction is an accelerator, not a gate
United outage or timeout	The vendor tier is dropped from that part’s sources; own-stock options still return — the request does not fail
Model throttling	Retry with backoff and batched pacing at index time; request-time timeouts bound prediction latency
SQL Server slow / replica crash	Brief caching serves last-known stock where safe; stateless replicas are replaced behind the load balancer

12 Non-goals & PII

Non-goals

- **A** — not route optimisation (it ranks who answers next, not a multi-stop route); deterministic, no LLM; the manual dispatch flow is untouched with the toggle OFF; no UI change beyond the toggle and its configuration; no schema change beyond the dispatch status fields (§7A: assigned techId, assignment fence token, assignment source, timestamps, and the append-once assignmentDecision audit document).
- **B** — does not fill gaps with invented text — a missing block degrades to an explicit MISSING placeholder, never an invented one, and the §9 groundedness eval (FND-024) gates releases on measured fabrication rate; never blocks Save (every failure mode degrades to the manual recap); completeness over prose — the

headline is cross-checking the note against parts, time and photos actually logged.

- **C** — no price ranking, no ordering (the technician decides where to collect); one supplier for the POC (United, via the existing CP gateway); predictions are suggestions, never auto-committed; no purchasing, PO creation, or inventory management — stock stays owned by the system of record.

PII inventory & data handling

Area	Control
Client → service auth	Bearer token from the app session, validated at the .NET API before the AI proxy forwards; CORS restricted to the mobile client
Network isolation	The AI layer runs in a private Azure VNet; in the target posture (VPN / VNet peering) it is never internet-reachable — only the .NET proxy (and private paths to stores and models) can reach it. If the pilot runs on the interim locked-down public HTTPS endpoint instead, that is the documented pilot posture: mTLS + IP allow-list, with WAF, per-source rate limits, client-certificate rotation, alerting on non-allow-listed source attempts, and a dated migration deadline and owner (<DATE / OWNER — to confirm>). Pilot posture is decided before build (§4, FND-004, Open Item #6)
Model hosting	Embedding and LLM inference run on client-hosted models. In the Prod privacy-first (default) configuration, dispatch text and parts history never leave client infrastructure and are never sent to a third-party AI provider. Dev is the stated exception (FND-041): it runs Gemini-only against a de-identified / synthetic dataset (§4), so no production-derived customer data reaches Google; any Gemini processing of ClimatePros-derived data requires the Open Item #9 DPA basis. The Google Gemini fallback is off by default; it may only be enabled after a documented ClimatePros data-use / DPA decision (region, retention, legal basis), and while disabled no data can reach it (Open Item #9)
Supplier gateway	United calls carry a subscription key held server-side only — it never reaches the mobile client
Secrets	All credentials resolved from Azure Key Vault with rotation — none in source or images

Area	Control
Data store access	Least-privilege, read-only SQL account scoped to inventory and dispatch tables; every AI write goes to the AI-owned MongoDB
Transport	TLS on all hops

- Dispatch and customer detail is served to the authenticated technician only and excluded from log bodies; van/warehouse locations are used transiently for distance ordering, not persisted.
- **PII / pricing scrubbing:** a deterministic, non-AI code layer on the FastAPI service strips customer PII and all pricing from every model call — the same on any runtime, primary or fallback — and re-inserts held values into the office-facing output only, so the model never sees customer data.

Scrub specification (FND-033) — Mechanism: the primary control is a field allow-list over the structured wizard payload — only explicitly allow-listed fields can enter a prompt; anything not listed is dropped by default, so new fields fail closed. Free-text portions (technician notes, dispatch reason) are scrubbed deterministically, consistent with the non-AI design of this layer, by (a) exact and normalised matching of the values already known from the job's structured records — customer and contact names, site and street addresses, phone numbers, e-mail addresses, part prices, labour/overhead rates, internal part numbers — and (b) pattern classes for phone numbers, e-mail addresses, street addresses and currency amounts. Entity classes covered: person/contact names (as known values), addresses, phone numbers, e-mail addresses, monetary amounts and rates, internal part numbers. Tokens and re-insertion: each removed value is replaced by an indexed placeholder token (`{{PII:class:n}}`); the token→value map lives in memory for the life of the request only and keys the re-insertion into the office-facing output. Failure behaviour: fail-closed — on any scrub error or timeout the model call is not made; B degrades to the manual recap and C returns 503 → manual add (§11). Validation: a CI leakage eval runs a labelled corpus of realistic technician notes through the scrub and asserts zero leaks per entity class; it gates releases exactly as the B eval set does (§9/§11), and the corpus grows with every observed near-miss. Audit: every `llm_calls` record carries a `scrubOutcome` field (classes found, counts, pass / fail-closed) so the leakage posture is auditable per call (§7B). Residual risk, stated: a free-text name absent from the structured record can evade value-matching; the pattern classes and

the growing CI corpus are the compensating control, and the leakage eval's zero-leak assertion is the measured false-negative target.

- **A:** the push token / endpointArn is the one new piece of personal data — encrypted at rest, access-limited, overwritten on rotation (§11).
- **B:** notes and the correction history carry the same sensitivity as today's manual notes — stored in the AI MongoDB behind the proxy. Unlike C's rebuildable index, they are original records: a cascading-deletion mechanism against SQL-side deletion requests still needs defining (Appendix).
- **C:** the vector store holds only dispatch reason text and used part numbers — derived from SQL Server and re-seedable at any time, so deletion/retention requests are handled at the SQL layer.

The history B and C learn from: 10–12 years of operational data — 200,000+ jobs in the last six months, 16.3M note records — and messy by the client's own admission (~40% of notes $\leq$ 40 characters). The corpus is de-duplicated and PII-stripped before anything reaches a model — a data-preparation step upstream of both §6 pipelines.

Changelog — v1.2 → v1.3 (per finding)

Each row maps one review finding to the sections touched and the change made. Rule 3 of the review brief is honoured throughout: no real-world value was invented — every unmeasured or unconfirmed figure is a clearly labelled <placeholder — to confirm/measure> carried in the Appendix Open Items. The companion Stream 1 Scope Document was not altered; where architecture and scope disagreed, the architecture was changed to match the scope.

Finding	Sections	Change
FND-050 (Blocker)	§6A, §9, §10	Travel-time provider pinned to Azuga (nearestvehicles) in the Step-A pseudocode; first-class §10 line — elements/dispatch, dispatches/day, cache, ~\$0 marginal cost with the metered ~\$4,000/mo alternative shown; caching design, circuit breaker, cost ceiling and decision record added; Open Item #11.
FND-005	§3, §4	Model identity and serving facts specified (tag/revision, params, quantisation, GPU/VRAM, vLLM limits, measured tokens/sec → max req/min) as labelled placeholders under

Finding	Sections	Change
		Open Item #1; declared the input to the §4 load test, §10 capacity and §8 admission control.
FND-006	§3, §4, §11A	Scaling unit stated: one Fargate task per enabled board (Open Item #12); §11A recovery rewritten to XAUTOCLAIM with idle threshold and expected recovery time; two-task option noted with §10 impact.
FND-038	How-to-read, §2, §3, §4, §9	On-prem tier added as a named third zone with its own connectivity row (mechanism placeholder, auth, failure behaviour, monitoring, ownership); “single cross-cloud hop” corrected everywhere; round-trip added to the P95 budget.
FND-039	§4, §11	Terraform committed as the IaC tool — one workflow/state across both clouds, remote state + locking, per-environment variable files, Key Vault / IAM secret references; Phase-0 gate restated as terraform apply + health checks.
FND-041	§4, §11, §12	Dev dataset changed to de-identified / synthetic (never a raw prod snapshot); Gemini-in-Dev tied to the Open Item #9 DPA basis or a local Dev model (Open Item #13); §12 guarantee made precise to the Prod configuration.
FND-051	§3, §4, §6A	Azuga named in the §3 stack and §4 infrastructure; A given the same degraded mode as C (distance fallback, flagged), plus per-lookup timeout, cache and circuit breaker; cross-referenced to FND-050.
FND-009	§5, §7A, §11A	Lock-renewal heartbeat and monotonic fence token added; .NET assignment endpoint rejects lower tokens (durable home = dispatch status fields); exactly-once restated as conditional; §11A slow-vs-crashed rows made genuinely distinguishable.
FND-054	§7A, §8A	Late-accept contract defined — 409 with offer_expired / already_assigned; client card expires locally on the pushed window deadline; outcomes recorded in the FND-035 decision record.
FND-011	§4, §7C, §10	Seeding contradiction resolved into an explicit decision (filtered subset vs full corpus) with placeholders and Open

Finding	Sections	Change
		Item #3; index type/parameters to pin at store selection; §10 vector line marked superseded pending the decision.
FND-022	§4, §7C	part_predictions collection defined (neighbours, distances, threshold gate, scores, confidence, usedParts); §4 collection list reconciled — §7 authoritative.
FND-035	§4, §7A, §12	Assignment-decision record persisted per dispatch (candidates, scores, w, fallback flag, offers/outcomes, final assignment) via the .NET API into the dispatch status fields; DR note updated.
FND-014	§8	Companion spec named and versioned (Streaml-AI-API.openapi.yaml v1.0.0, OpenAPI 3.1, all 14 routes, error codes, auth); pydantic models + client generated from it; crossTierBufferMinutes typed (integer minutes, nullable, default null = never; range to confirm).
FND-016	§4, §8B	Admission control at the AI service — semaphore sized to FND-005 capacity, bounded queue with max wait, fast 429/503 + Retry-After, per-technician/per-submission caps; Container Apps scales on queue wait.
FND-028	§6C, §7C, §8C, §9	Completion push promoted to a specified internal endpoint (POST /parts/completions — payload, auth, retries, idempotent upsert by dispatchNo); trigger options stated (sync vs outbox, Open Item #4); index-growth monitor added.
FND-023	§4, §9	Provisional P95 targets set from figures already in the document, gating Phase 3; full alert table added (parse failures, model errors/timeouts, queue depth, Redis pending age, Cosmos headroom, proxy 5xx, usage cliff, index growth, geo spend, tag divergence, edit-rate drift).
FND-024	§1, §9, §12	Groundedness eval added (claim-to-source tracing, fabrication rate per section, LLM-judge validated against human labels); §1/§12 “never fabricates” softened to the mechanism in place (MISSING placeholder) plus the eval gate.

Finding	Sections	Change
FND-025	§9	Release gate made arithmetic — target/minimum per metric (set at baseline, Open Item #14), max regression (2 pts/section, aggregate may not worsen — proposed), eval-set size floor (200 — proposed) failing closed; B eval set seeded pre-go-live from hand-corrected stratified historical closeouts.
FND-026	§9	Engagement-independent quality signal added (supervisor/QA rubric sample + groundedness); time-on-review-screen tracked with a disengagement flag; edit-rate segmented per technician.
FND-029	§6C	Index admission criteria stated at the upsert — min reason length/information threshold, exclusion of returned/credited/corrected parts and the backtest hold-out, down-weighting of unedited accepted predictions; negative signal pulled forward where feasible.
FND-033	§6B, §7B, §12	Scrub given a full specification — field allow-list primary, deterministic free-text technique with stated entity classes, token scheme + keyed re-insertion, fail-closed behaviour, CI leakage eval gating releases, scrubOutcome audit field on llm_calls.
FND-004	§4, §11B, §12	The three contradictory postures reconciled into one decision (Open Item #6): target VPN/peering keeps the absolute claim; the interim endpoint, if used, becomes the documented §12 posture with compensating controls and a dated migration deadline + owner.
FND-052	§7, §11	Migration stage added before app deploys — versioned idempotent forward-only Mongo runner with N/N-1 reader rule; SQL dispatch status fields sequenced expand→migrate→contract; vector index changes via dual-index build-then-swap behind the rollout flag.
FND-053	§3, §7B, §9, §11	On-prem model tier added to release management — pinned tags at the gateway, named deployer, one-step re-pin revert, served-tag recorded and divergence-alerted; combined embedding-change procedure (eval → parallel index → backtest → swap → retained rollback index).

Finding	Sections	Change
Replica reconciliation	§4, §10, §11B	“Min 1 replica” vs “min 2 always-on” resolved to min 2 everywhere; §4 compute sizing updated and the §10 Container Apps floor re-derived (~\$40/mo) with scenario totals updated.
Close-out	Doc control, Appendix	Version bumped to 1.3; Open Items #1/#3/#4/#6/#9 updated and #11–#14 added; Scope Document unchanged.

Appendix — Open Items

Each needs a ClimatePros owner before this document moves from draft to agreed.

#	Open item	Where	Why it matters
1	Private-runtime / model-gateway readiness — endpoint, credentials, model version, first integration-test date; now also carries the FND-005 serving facts (exact model tag/revision, param + active-param count, quantisation, GPU model/count + VRAM, vLLM max-model-length + max-concurrent-sequences, measured tokens/sec → max sustained requests/minute) and the FND-038 Azure→on-prem network mechanism (ExpressRoute / site-to-site VPN / authenticated endpoint; bandwidth, latency)	§2–§4	The privacy-first posture depends on it; capacity figures are client-stated
2	Azure re-pricing for the AI lines (Container Apps, vector store, Monitor)	§10	Current figures are AWS-sourced rates
3	Vector store selection (Azure AI Search / Pinecone) and its C-only scope; plus the seeding decision — filtered subset vs the full 200k+/6-month corpus (FND-011) — the index parameters (HNSW m/ef or the managed	§3, §6	C needs a store at go-live; B only if evals justify it

#	Open item	Where	Why it matters
	equivalent), and the re-derived §10 line		
4	Learning-loop trigger for C – the endpoint is now specified (§8C POST /parts/completions, FND-028); still open: the .NET-side trigger mechanism (synchronous on completion vs outbox + scheduled drain) and its owner	§6, §7	Without it the self-learning index never updates
5	MongoDB collection and index design sign-off with the client DBA	§7	Query performance for retrieval and the eval dashboard
6	Cross-cloud network path – VPN / peering target vs. interim allow-listed endpoint; now includes the pilot-posture decision with compensating controls and a dated migration deadline + owner (FND-004), and cross-references the Azure→on-prem seam under #1 (FND-038)	§4	Security review and the P95 budget both depend on it
7	Push token / endpointArn in the PII inventory and retention policy	§11, §12	The one new piece of personal data this project stores
8	Embedding model – pinned to bge-m3 (1024-dim dense), the sole model behind the LiteLLM gateway; earlier nomic-embed references are superseded. Confirm at sign-off.	§3, §4	Resolved to bge-m3; dimensionality, cost, and capacity estimates key off it.
9	Gemini fallback ships OFF by default; §4/§12 updated so the third-party-provider claim holds while it is disabled – and §12's guarantee is now stated for the Prod configuration. Open decision: whether ClimatePros wants it enabled at all, and if so the data-use / DPA terms (region, retention, legal basis); the same basis	§3, §12	PII posture stays consistent while the enablement decision is pending.

#	Open item	Where	Why it matters
	governs any Gemini processing of ClimatePros-derived data in Dev (FND-041).		
10	Cascading deletion for B's summaries/edits against SQL-side deletion requests	§12	They are original records, not rebuildable
11	Azuga contract coverage — confirm the existing fleet-telematics contract covers the auto-dispatch travel-time lookup volume (~28,000/day pre-cache); the ~\$0 marginal-cost line rests on it (FND-050/051)	§3, §4, §6, §10	The blocker's cost model rests on it
12	Worker scaling unit — confirm one Fargate task per enabled division/board (FND-006); if concurrent redundancy is required, raise the floor to two tasks per board and re-derive the §10 Fargate line	§4, §11	The XAUTOCLAIM recovery path and the Fargate cost both key off it
13	Dev dataset & Dev model — build the de-identified / synthetic Dev dataset and decide Dev's model path (small quantised local model vs Gemini under the #9 DPA basis) (FND-041)	§4, §11, §12	One environment otherwise sends prod-derived data to a third party
14	Eval-gate values & B seed set — ratify the FND-025 gate numbers at the pilot baseline and complete the supervisor-labelled stratified seed set before go-live	§9	The release gate fails closed without them